OS & SP

Name : D. Teja Sadwika **Roll No :** 2520030536

SKILL – 19

1.Skill19.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

/*
 * SIGINT handler
 * Ctrl+C sends SIGINT to the shell.
 * The shell forwards it to the foreground child.
```

```
 */
void handle_sigint(int sig)
{
    (void)sig;

    if (foreground_pid > 0) {
        printf("\nShell: Forwarding SIGINT to foreground
process\n");
        kill(foreground_pid, SIGINT);
    } else {
        printf("\nShell: Ignoring SIGINT\n");
    }
}

/* SIGTERM handler */
void handle_sigterm(int sig)
{
    (void)sig;
    printf("\nShell received SIGTERM\n");
```

```
}

int main(void)
{
    char input[256];

    /* Register SIGINT handler */
    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction SIGINT");
        return 1;
    }
```

```
/* Register SIGTERM handler */
struct sigaction term_action;

memset(&term_action, 0, sizeof(term_action));
term_action.sa_handler = handle_sigterm;
sigemptyset(&term_action.sa_mask);
term_action.sa_flags = SA_RESTART;

if (sigaction(SIGTERM, &term_action, NULL) == -1) {
   perror("sigaction SIGTERM");
   return 1;
}

printf("====================================\n");
printf(" OSSP Program 19 - Signal Handling\n");
printf("====================================\n");
```

```
printf("Commands:\n");
printf(" Any Linux command -> execute\n");
printf(" exit            -> exit shell\n\n");

while (1) {
    printf("shell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        continue;

    /* Exit shell */
    if (strcmp(input, "exit") == 0)
```

```
    break;

/* Create child process */
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    continue;
}

/* Child process */
if (pid == 0) {
    /* Restore default signal behavior */
    signal(SIGINT, SIG_DFL);
    signal(SIGTERM, SIG_DFL);

    char *args[64];
    int argc = 0;
```

```
char *token = strtok(input, " \t");

while (token != NULL && argc < 63) {
    args[argc++] = token;
    token = strtok(NULL, " \t");
}

args[argc] = NULL;

if (argc == 0)
    _exit(0);

/* Execute command */
execvp(args[0], args);

perror("execvp");
_exit(1);
```

```
}

/* Parent process */
else {
    foreground_pid = pid;
    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid");
    } else {
        foreground_pid = -1;
        if (WIFEXITED(status)) {
            printf("Process exited with status %d\n",
                WEXITSTATUS(status));
        }
        else if (WIFSIGNALED(status)) {
            printf("Process terminated by signal %d\n",
                WTERMSIG(status));
        }
```

```
        }
      }
    }
    printf("\nShell terminated.\n");
    return 0;
}
```

OUTPUT :

